

Agenda: Build First MCP Server and Client

- Setup - SDKs & FastMCP
- MCP CLI and package installation
- Stdio logging rule
- Hands-on lab setup
- Create and run the MCP server
- Verify using Inspector CLI
- Open and use the Inspector UI
- Use the server from Claude Desktop
- Write a Python client for the MCP server

Setup – MCPServer SDK

Official SDKs are tiered.

Tier	Languages
Tier 1	TypeScript · Python · C# · Go
Tier 2	Java · Rust
Tier 3	Swift · Ruby · PHP

Python is Tier 1 ([modelcontextprotocol/python-sdk](#), package [mcp](#)).

Within it, **MCPServer** / **FastMCP** is the high-level decorator API — the fast path for tools/resources/prompts.

- ▶ Type hints → inputSchema — `a: int` becomes `{"type": "integer"}`
- ▶ Docstring → tool description — written once, surfaced to every client
- ▶ Return value → tool result — FastMCP wraps it as `content[]`
- ▶ Writing a Python function with type hints + docstring IS defining the protocol surface

```
@mcp.tool()
```

```
def multiply(a: float, b: float) -> float:
```

```
    """Multiply two numbers together and return the result."""
```

```
    return a * b
```

Following will install MCP Package and MCP Cli Tools

```
pip install "mcp[cli]"
```

MCP CLI:

- **mcp dev server.py** (`npx @modelcontextprotocol/inspector python server.py` — your development/debugging command. It launches the server together with the MCP Inspector. It

also supports on-the-fly dependencies and live-reloading of local code: **mcp dev server.py --with pandas --with numpy** adds packages, and **--with-editable .** mounts your local code so changes show up without reinstalling.

- **mcp run server.py** — runs the server directly, as a plain process. **No Inspector**, no dependency management — it just imports your server object and starts it. The catch is that you're responsible for making sure all dependencies are available in the environment. Functionally this is close to just running **python server.py** yourself (assuming you have the `if __name__ == "__main__": mcp.run()`), so it's mostly a convenience wrapper.

The stdio logging rule (non-negotiable)

Your first servers use **stdio** (host launches the process; JSON-RPC over stdin/stdout). Therefore: **STDIO servers must never write to stdout** — *it carries the JSON-RPC messages; any stray write corrupts the protocol and the server dies "silently". (HTTP servers: stdout is fine.)* — **mcp-**

```
import sys, logging

print("hi")                # ✗ corrupts a stdio server
print("hi", file=sys.stderr) # ✓
logging.info("hi")         # ✓ (stderr/file)
```

A silent or instantly-dying first server is almost always a stray **print()**/banner on stdout. This is *the* classic bug.

Hands-On Lab

Prereqs: Python 3.10+ (**python --version**), **uv** (**uv --version**), Node 18+ (**node --version**, for the Inspector).

Setup: Install **uv** then restart your terminal:

```
# macOS/Linux
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
# Windows
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Step 1 — Scaffold

```
mkdir mcp-first-demo
cd mcp-first-demo
python -m venv venv
venv/Scripts/activate #Windows
pip install "mcp[cli]"
OR
uv init first-mcp-demo
cd first-mcp-demo
uv add "mcp[cli]"
```

- `.venv\Scripts\activate` on Windows PowerShell,
- `source .venv/bin/activate` on macOS/Linux.

Step2 - Create server.py

```
from mcp.server.mcpserver import MCPServer

# Create an STDIO/HTTP MCP server
mcp = MCPServer("Demo")

# Add an addition tool
@mcp.tool()
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b

@mcp.tool()
def subtract(a: int, b: int) -> int:
    """Subtract two numbers"""
    return a - b

@mcp.tool()
def multiply(a: float, b: float) -> float:
    """Multiply two numbers together and return the result."""
    return a * b
```

```

@mcp.tool()
def divide(a: float, b: float) -> float:
    """
    Divide a by b and return the result.
    Raises: ValueError: If b is zero
    """
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b

# Add a dynamic greeting resource
@mcp.resource("greeting://{name}")
def get_greeting(name: str) -> str:
    """Get a personalized greeting"""
    return f"Hello, {name}!"

# Main execution block - this is required to run the server
if __name__ == "__main__":
    mcp.run(transport="stdio") #local
    #mcp.run(transport="streamable-http") #remote

```

Note:

- The one thing to see here: `greeting://{name}` has a `{placeholder}`, so it's a **resource template** — reading `greeting://Ada` returns `Hello, Ada!`.
- The one thing to see: `divide` raises `ValueError` on zero — FastMCP converts the exception

Step 3 - Run it:

```
uv run server.py
```

Note: It "hangs" waiting for JSON-RPC on stdin; Use **Ctrl+C** to stop).

Step 4 – Install Inspector

First Install latest version of Node

```
npm install --global @modelcontextprotocol/inspector@latest
```

Verify with the Inspector CLI:

```
npx @modelcontextprotocol/inspector --cli uv run server.py --method tools/list    # Python  
executes the file directly
```

OR

```
npx @modelcontextprotocol/inspector --cli uv run mcp run server.py --method tools/list    #The  
MCP CLI imports the file, finds its server object, and calls .run()
```

Note: The second command runs for both studio and streamable-http.

```
npx @modelcontextprotocol/inspector --cli uv run server.py --method tools/call --tool-name add --  
tool-arg a=1 --tool-arg b=2
```

```
npx @modelcontextprotocol/inspector --cli uv run server.py --method tools/call --tool-name divide -  
-tool-arg a=1 --tool-arg b=0
```

The result comes back with `"isError": true` — the server keeps running.

Opens the Inspector UI

Step1: Launch the Inspector UI

```
mcp dev server.py
```

OR

```
npx @modelcontextprotocol/inspector python server.py
```

OR

Add to Workspace pyproject.toml

```
[build-system]  
requires = ["setuptools>=68"]  
build-backend = "setuptools.build_meta"  
  
[project]  
name = "mcp-server-demo"  
version = "0.1.0"  
description = "Demo MCP server"  
requires-python = ">=3.10"  
dependencies = [  
    "mcp>=1.0.0",  
]
```

```
[tool.setuptools]
py-modules = ["server"]
```

then RUN

```
mcp dev server.py --with-editable d:/demos/mcp-server
```

Edit the server.py (change anything), Refresh the browser and note the change is reflected without restarting server.py

Step 4: Connect and test tools and Resource Template

To Debug MCP Server with Break point:

Change the main as below:

```
if __name__ == "__main__":
    #mcp.run(transport="stdio") #local
    mcp.run(transport="streamable-http") #remote
```

In Terminal 1:

```
python server.py
```

In Terminal 2:

```
npx @modelcontextprotocol/inspector
```

In Browser: Goto MCP Inspector → Add Servers → Add Manually:

Add server ✕

Server ID *
Used as the key in mcp.json. Letters, numbers, hyphens, underscores.

my-server

Transport

streamable-http ▾

URL *

http://127.0.0.1:8080/mcp ✕

Cancel Add

Note: Url should be <http://localhost:XXXX/mcp>

Connect and test the method. Put a break point in Python Code to break at that point while calling the method from MCP client in MCP Inspector.

Use Server from Claude Desktop

The following command registers the server into the Claude Desktop config file so the app can launch it automatically.

```
mcp install server.py
```

OR

Manually Add to `claude_desktop_config.json`

```
"mcpServers": {  
  "Demo": {  
    "command": "uv",  
    "args": [  
      "run",  
      "--frozen",  
      "--with",  
      "mcp[cli]==2.0.0",  
      "mcp",  
      "run",  
      "D:\\Demos\\mcp-server\\server.py"  
    ]  
  }  
}
```

Writing a Python Client to use the MCP Server.

stdio_client.py

```
import asyncio  
import sys  
  
from mcp import ClientSession, StdioServerParameters  
from mcp.client.stdio import stdio_client  
  
async def main() -> None:  
    # Launch the local MCP server over stdio.  
    server_params = StdioServerParameters(  
        command="python",
```

```

    args=["server.py"],
)

async with stdio_client(server_params) as (read_stream, write_stream):
    async with ClientSession(read_stream, write_stream) as session:
        await session.initialize()

        result = await session.call_tool("add", {"a": 5, "b": 7})
        print("add(5, 7) =", result.content[0].text)

        greeting = await session.read_resource("greeting://Alice")
        print("greeting://Alice =", greeting.contents[0].text)

if __name__ == "__main__":
    asyncio.run(main())

```

Key difference: tools return **.content**, resources return **.contents** (plural).

If Streamable HTTP is used:

http_client.py

```

import asyncio

from mcp import ClientSession
from mcp.client.streamable_http import streamable_http_client

async def main() -> None:
    # Default FastMCP streamable-http endpoint is /mcp.
    async with streamable_http_client("http://localhost:8000/mcp") as (
        read_stream,
        write_stream,
        _get_session_id,
    ):
        async with ClientSession(read_stream, write_stream) as session:
            await session.initialize()

            result = await session.call_tool("add", {"a": 5, "b": 7})

```



```
print("add(5, 7) =", result.content[0].text)

greeting = await session.read_resource("greeting://Alice")
print("greeting://Alice =", greeting.contents[0].text)

if __name__ == "__main__":
    asyncio.run(main())
```

The pattern is identical — just swap the transport context manager. The HTTP client hits /mcp by default on the streamable-http server.

Terminal 1: Run the Server

```
uv run server.py
```

Terminal 2: Run the Client

```
uv run http_client.py
```